Security Audit Report

Project Title

SecureCode Audit: Security Review and Remediation of a Python Authentication System

Author

Nirupa P

1. Introduction

Authentication systems are a primary target for cyberattacks because they control access to sensitive resources. Weak authentication mechanisms can lead to unauthorized access, credential theft, and brute-force attacks.

This project focuses on auditing a vulnerable login application, identifying security weaknesses, and implementing remediation measures using secure coding practices.

2. Project Objective

The objectives of this project are:

- Identify vulnerabilities in a login application.
- Analyze the security risks associated with each vulnerability.
- Implement appropriate security controls.
- Compare the vulnerable and secure versions of the application.
- Demonstrate secure coding practices.

3. Vulnerability Assessment

Vulnerability 1: Hardcoded Credentials

Description:
The username and password were directly stored within the source code.

Risk Level: High

Impact:
If an attacker gains access to the source code, valid credentials become immediately visible.

Vulnerability 2: Plaintext Password Storage

Description:
Passwords were stored and compared as plain text.

Risk Level: Critical

Impact:
Compromised source code would expose user credentials directly.

Vulnerability 3: Unlimited Login Attempts

Description:
The application initially allowed unlimited authentication attempts.

Risk Level: High

Impact:
Attackers could perform brute-force attacks to guess credentials.

Vulnerability 4: Weak Password Acceptance

Description:
The application accepted weak passwords without enforcing complexity requirements.

Risk Level: Medium

Impact:
Weak passwords significantly increase the likelihood of unauthorized access.

4. Security Controls Implemented

Password Hashing

SHA-256 hashing was implemented to secure password storage and verification.

Benefit:
Passwords are no longer stored in plain text.

Login Attempt Limitation

The application restricts users to three authentication attempts.

Benefit:
Reduces the effectiveness of brute-force attacks.

Password Strength Validation

Passwords are required to contain:

- Minimum 8 characters
- Uppercase letter
- Lowercase letter
- Numeric value
- Special character

Benefit:
Improves credential strength and resistance against password attacks.

5. Before and After Comparison

Security Feature	***Vulnerable Version***	***Secure Version***
Plaintext Password Storage	Yes	No
Password Hashing	No	Yes
Login Attempt Limiting	No	Yes
Password Complexity Rules	No	Yes
Brute Force Protection	No	Yes

6. Results

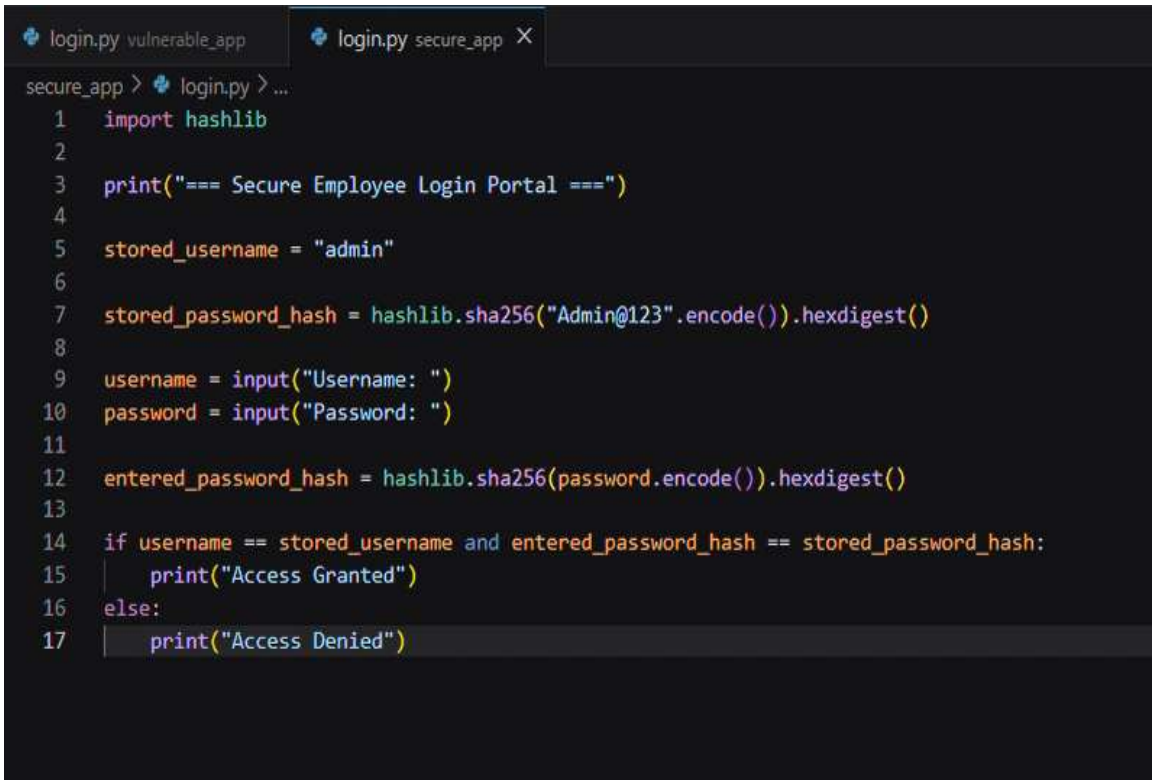
The secure version of the application successfully mitigated multiple authentication-related vulnerabilities.

Implemented controls significantly improved the security posture of the application by protecting credentials, enforcing password complexity, and reducing the risk of brute-force attacks.

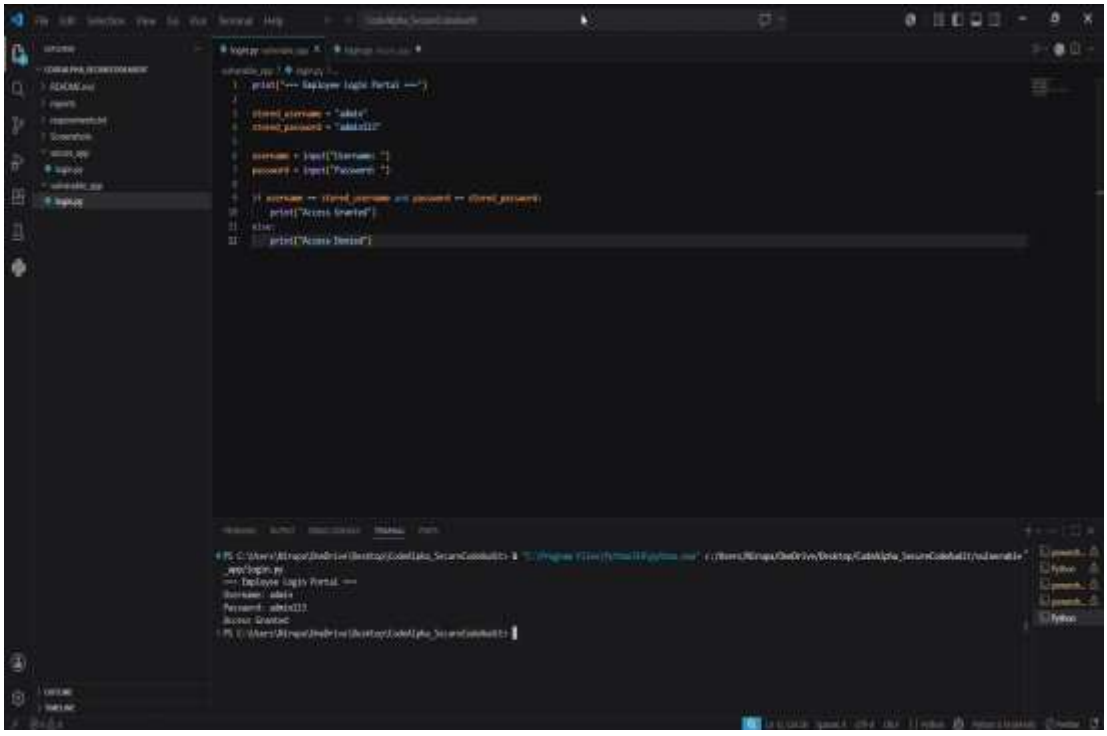
7. Screenshots

Include the following screenshots:

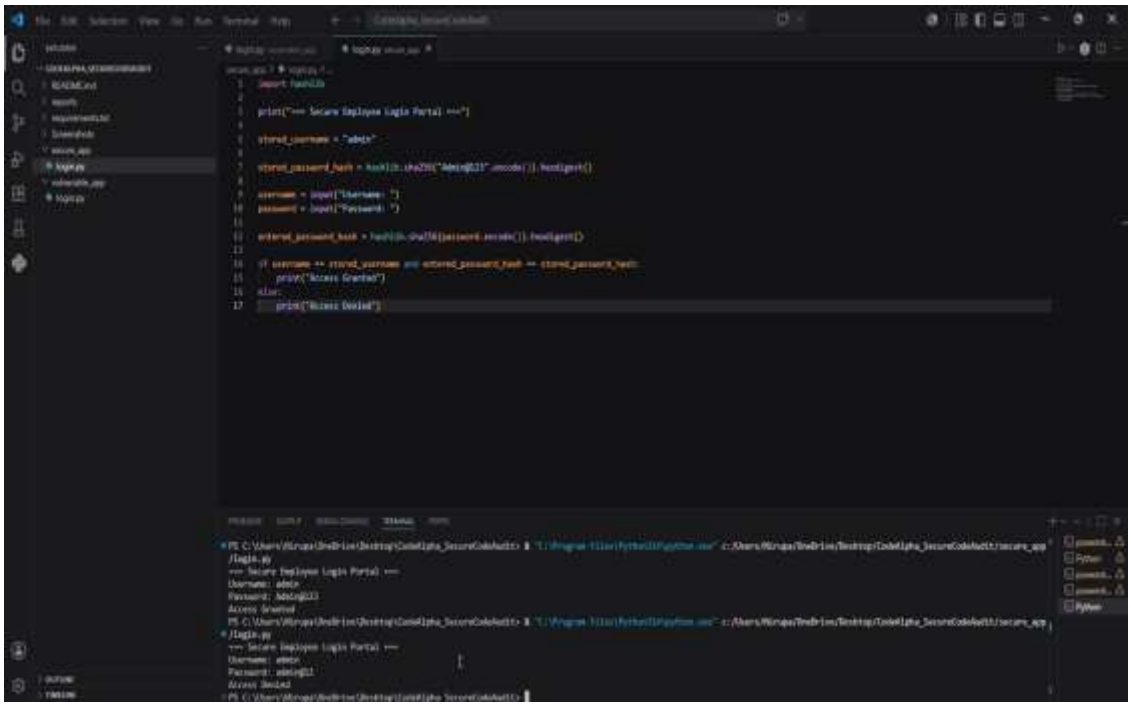
1. Source Code



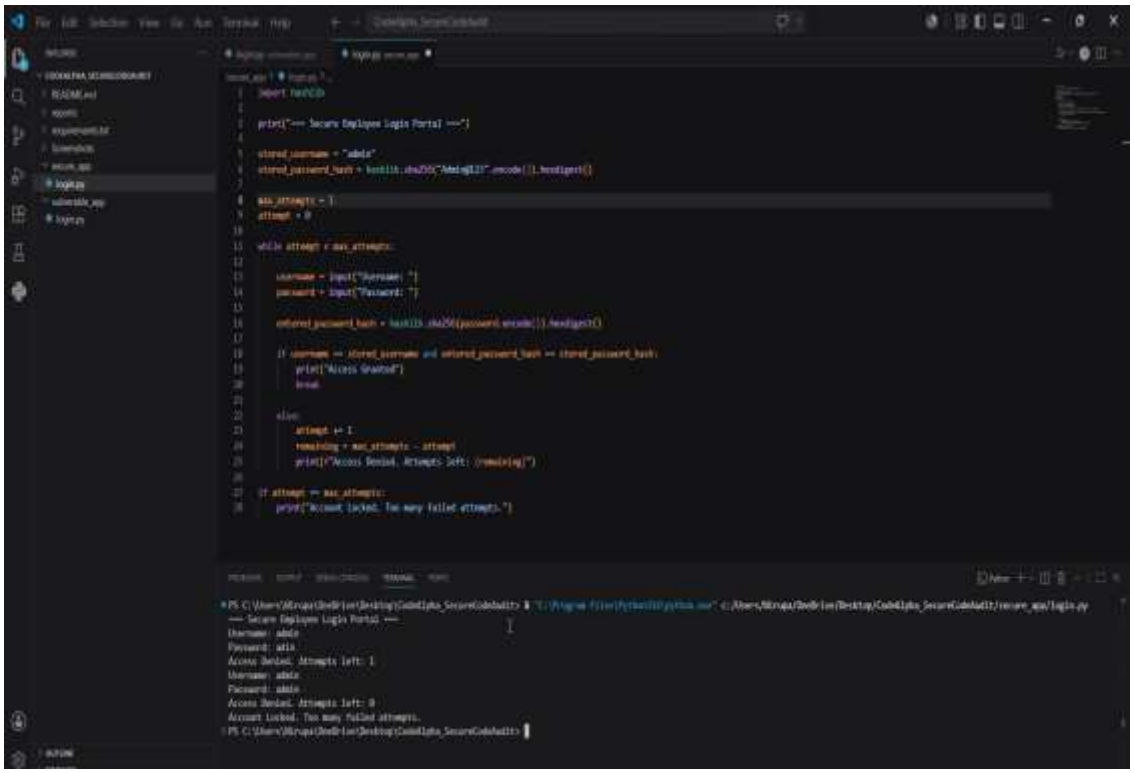
2. Successful Login

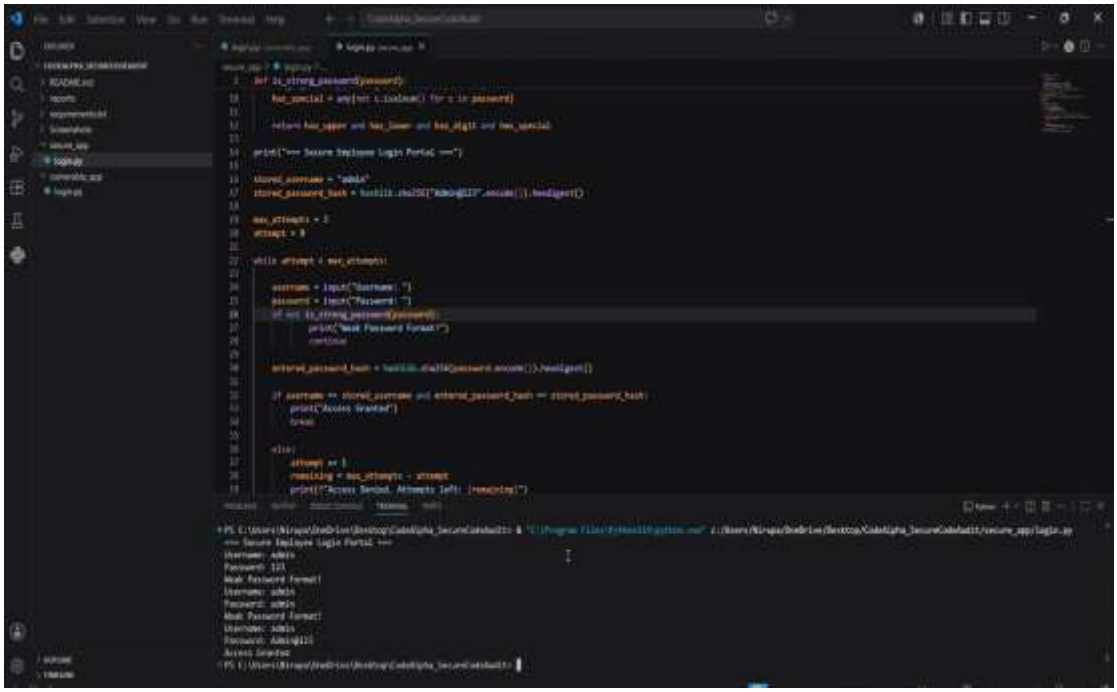


3. Failed Login



4. Account Locked After 2 Failed Attempts





8. Conclusion

This project demonstrated the process of identifying and remediating common authentication vulnerabilities in a Python-based login application.

The implementation of password hashing, brute-force protection, and password complexity validation improved the overall security of the system and provided practical exposure to secure coding principles and vulnerability assessment techniques.

The project highlights the importance of integrating security controls during software development to reduce risks and improve application resilience.